

SMART EXPENSE & BUDGET MANAGER

(A Command-Line Application for Personal Finance Tracking)

Submitted by: Shreeya Raj

Registration Number: 25BAI10515 {CSE AI & ML (2nd Year)}

Course: Programming in Java

Under the guidance of
Sharmila Joseph Ma'am

VIT Bhopal

1. INTRODUCTION

Personal finance management is one of the most consistently recommended life skills, yet most people struggle to actually practice it. Budgeting apps exist, but many of them require accounts, internet connectivity, and heavy graphical interfaces that are overkill for someone who simply wants to log a purchase and see how it affects their monthly budget.

This project presents the Smart Expense & Budget Manager, a command-line Java application that brings personal finance tracking down to its essentials. The user can record income and expenses, set monthly budget limits for spending categories, and view a colour-coded monthly report. All data is saved locally in simple CSV files, so the application works completely offline and requires no sign-up.

The project was developed as part of the Flipped Course Evaluated Course Project. It applies core object-oriented programming principles — inheritance, abstraction, encapsulation, and separation of concerns — in a small but realistic software system. The implementation uses only the Java Standard Library, keeping the project portable and dependency-free.

The rest of this report describes the problem the project addresses, its functional and non-functional requirements, the design decisions behind its architecture, implementation details, screenshots of the running application, and the challenges faced during development.

2. PROBLEM STATEMENT

Managing personal finances manually is inefficient and error-prone. Most people lose track of where their money goes within a few weeks, especially when spending is spread across many categories such as food, transport, and utilities. Spreadsheets help but require constant manual updating and give no immediate feedback when a category is close to or has exceeded its budget.

This project addresses that gap by providing a lightweight, terminal-based tool where a user can log transactions as they happen, set monthly budgets per category, and see at a glance whether they are within their limits. The application is designed to be:

- Fast — no waiting for a browser to load
- Offline — no internet connection required
- Portable — runs on any machine with a JDK installed
- Self-contained — no database, no sign-up, no configuration

3. FUNCTIONAL REQUIREMENTS

The application provides the following functionality:

FR1 — Add Income

The user can record an income entry with an amount, category, date, and description.

FR2 — Add Expense

The user can record an expense entry with an amount, category, date, and description.

FR3 — Set Monthly Budget

The user can define a monthly spending limit for a given category. Multiple budgets can exist for different categories and months.

FR4 — View Monthly Report

The user can view a report for any month, showing for each budgeted category:

- The budget limit
- The actual amount spent
- A status indicator (OK, WARN, or OVER)
- A visual progress bar showing budget usage

FR5 — Delete Transaction

The user can delete a transaction by its ID.

FR6 — Persistent Storage

All transactions and budgets are saved to local CSV files and automatically reloaded on the next run.

FR7 — Auto-Creation of Data Folders

The application automatically creates the data/ and docs/ folders on first run, so no manual setup is required.

FR8 — Input Validation

Every input is validated. Invalid numbers, malformed dates, and empty fields trigger a re-prompt without crashing the program.

4. NON-FUNCTIONAL REQUIREMENTS

NFR1 — Usability

The interface is text-based but uses colour-coded output (green for OK, yellow for warnings, red for overspending) and clear menu labels. A user who has never seen the application can use it within a few seconds.

NFR2 — Reliability

Input validation prevents the most common causes of crashes. The program always returns to the main menu after each action, so a single mistake does not end the session.

NFR3 — Portability

The project uses only the Java Standard Library and plain text files. It runs on Windows, macOS, and Linux with any JDK 17 or newer. No installation steps beyond having a JDK.

NFR4 — Maintainability

The code is organised into five layers (model, service, storage, util, ui), each with a single responsibility. Adding a new feature typically involves changes to only one or two classes.

NFR5 — Performance

All operations complete in milliseconds. The dataset for a single user is small enough that even linear scans remain effectively instant.

NFR6 — Data Integrity

Data is written to disk using a CSV format that is human-readable and can be inspected or corrected in a text editor if needed.

5. SYSTEM ARCHITECTURE

The application follows a layered architecture to separate concerns:

[INSERT IMAGE: 01_architecture.png]

The layers are:

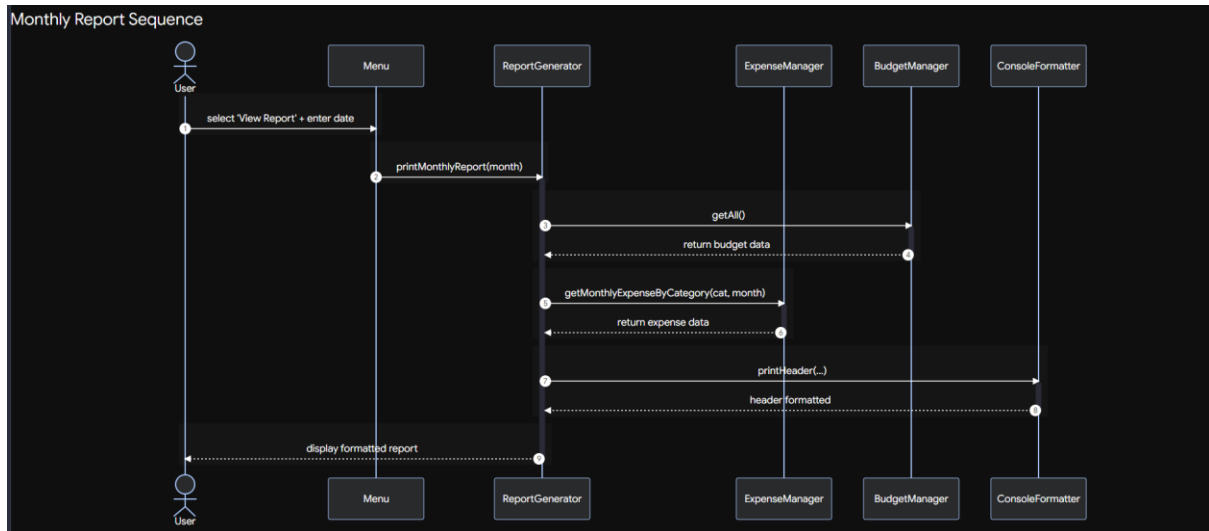
- Entry Point — Main sets up folders and launches the menu.
- UI Layer — Menu presents the console menu, reads user choice, and dispatches to services.
- Service Layer — ExpenseManager, BudgetManager, and ReportGenerator hold the business logic.
- Storage Layer — FileStorage reads and writes CSV files.
- Utility Layer — ConsoleFormatter handles output styling; InputValidator handles input parsing.
- Model Layer — Transaction (abstract base), Expense, Income, and Budget represent the domain data.

Calls flow downward: the UI calls services, services call storage, and all layers use the utility classes for formatting and input.

6. DESIGN DIAGRAMS

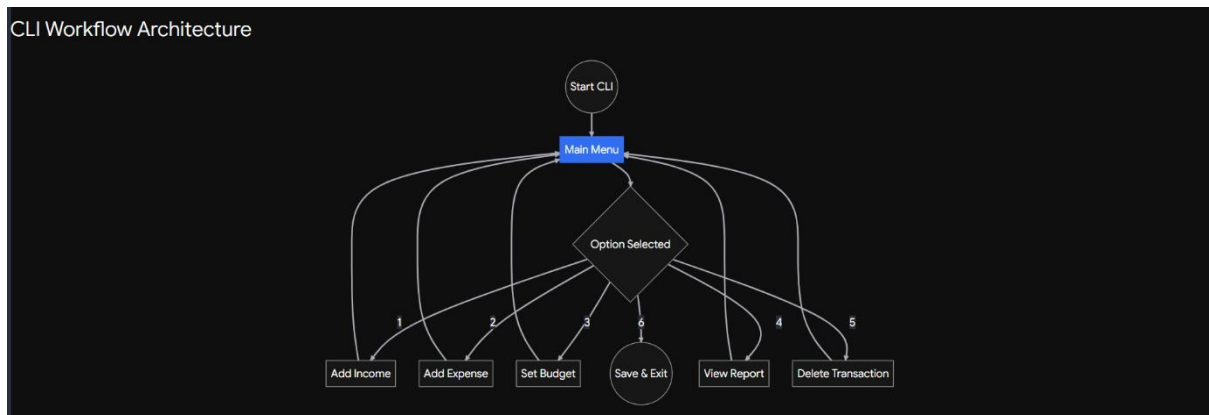
6.1 Use Case Diagram

Shows what the user can do with the system.



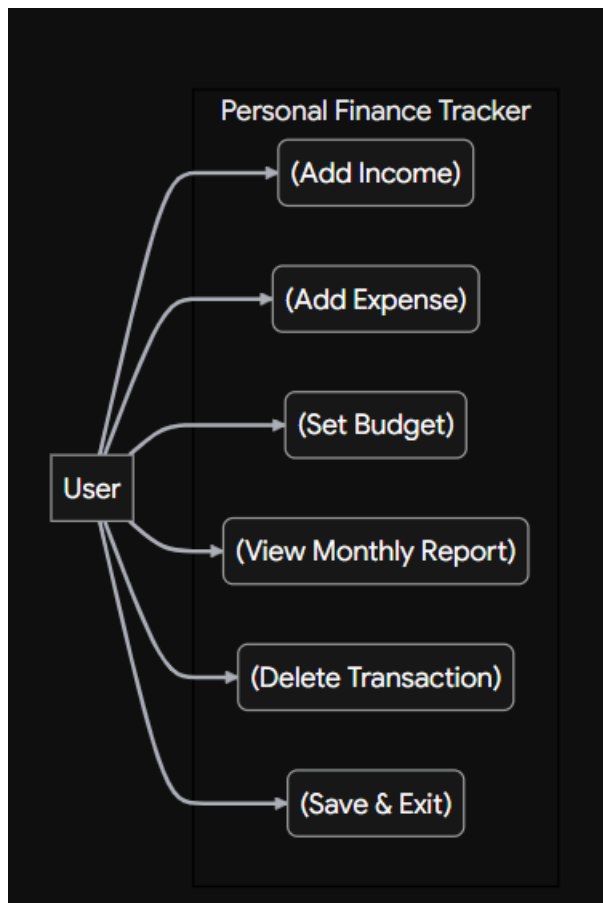
6.2 Workflow Diagram

Shows the flow of user actions through the main menu.



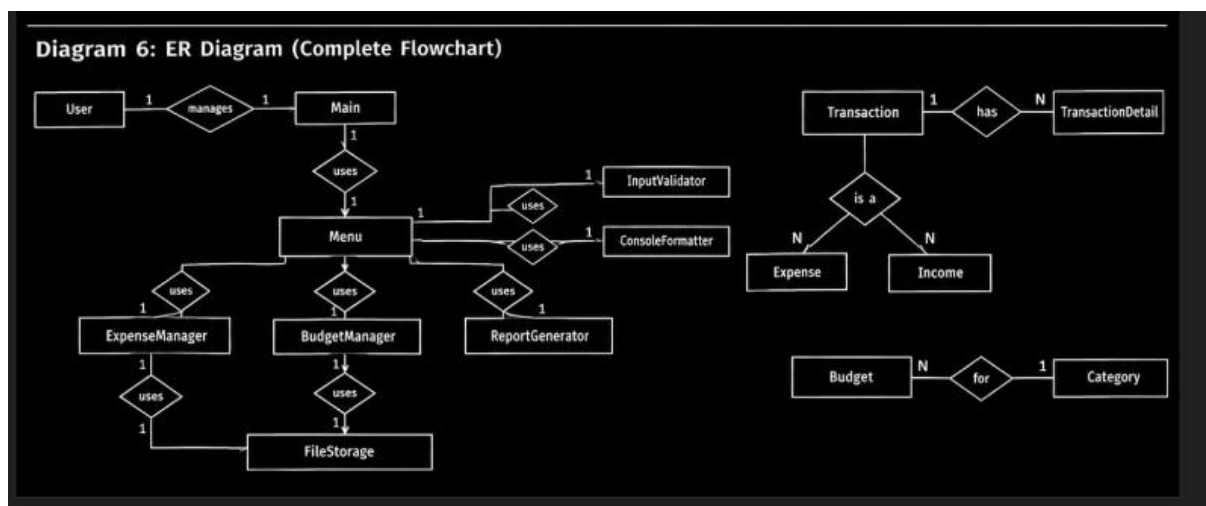
6.3 Sequence Diagram — View Monthly Report

Shows the exact sequence of calls when the user views a monthly report.



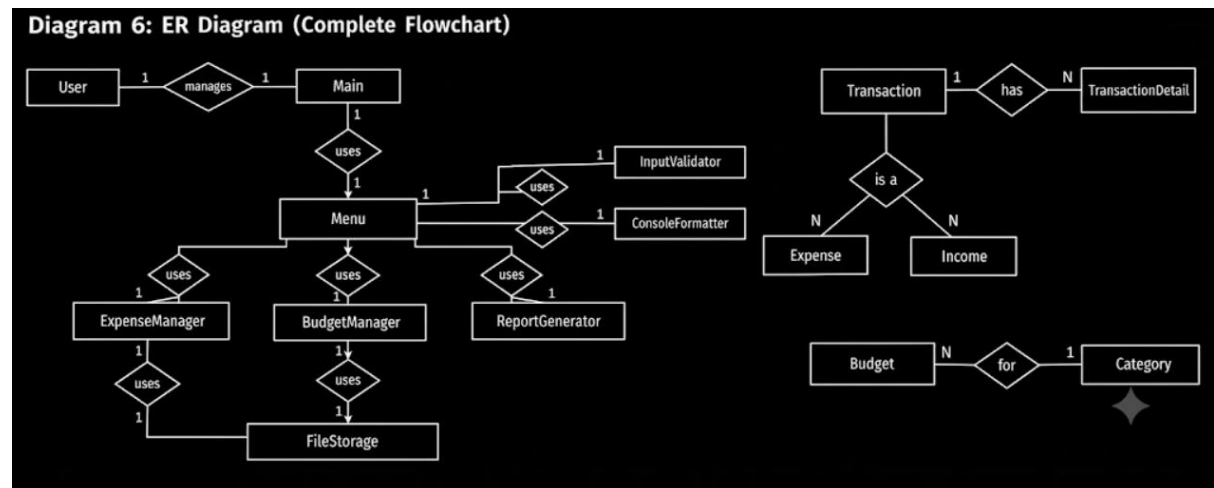
6.4 Class Diagram

Shows all classes in the project and their relationships.



6.5 Entity-Relationship Diagram

Represents the CSV-based storage schema. Two independent tables are used.



7. DESIGN DECISIONS & RATIONALE

Why a layered architecture?

Each layer has a single responsibility. The model layer knows nothing about how data is displayed. The storage layer knows nothing about the business rules. This makes the code easier to test, modify, and reason about.

Why an abstract Transaction class?

Income and expense entries share almost all of their structure — id, amount, category, date, description. The only difference is their type. Extracting a common base class removes duplication and makes it easy to add new transaction types (e.g., transfers) later.

Why CSV instead of a database?

A full database engine (SQLite, H2, etc.) would be overkill for a single-user offline application. CSV files are readable, editable, and require zero setup. If the project grew to require querying large datasets, swapping in SQLite would be straightforward because all storage access is already isolated inside FileStorage.

Why no external libraries?

The project intentionally depends only on the Java Standard Library. This makes compilation a single javac command and ensures the project runs anywhere without Maven, Gradle, or a package manager.

Why ANSI colours?

Colour-coded output makes it possible to scan the monthly report and immediately see problem categories. Green for OK, yellow for near-limit, red for over-budget. This is achieved with ANSI escape codes, which work in any modern terminal without external libraries.

Why a progress bar instead of just a number?

Percentages require reading and mental comparison. A bar chart () is understood at a glance.

8. IMPLEMENTATION DETAILS

8.1 Transaction Model

The Transaction class is an abstract base that stores the shared fields and exposes them via getters. A static counter assigns each transaction a unique ID as it is created.

```
public abstract class Transaction {  
    private static int counter = 1;  
    private final int id;  
    private final double amount;  
    private final String category;  
    private final LocalDate date;  
    private final String description;  
    private final String type;  
  
    public Transaction(double amount, String category, LocalDate date,  
        String description, String type) {  
        this.id = counter++;  
        this.amount = amount;  
        this.category = category;  
        this.date = date;  
        this.description = description;  
        this.type = type;  
    }  
  
    public String toCsv() {  
        return id + "," + type + "," + amount + "," +  
            category + "," + date + "," + description;  
    }  
}
```

}Expense and Income extend this class and only set the type constant in their constructors.

8.2 Expense Manager — Business Logic

Totals and per-category monthly sums are computed using Java Streams. This keeps the logic compact and eliminates loop boilerplate.

```
public double getTotalIncome() {
    return transactions.stream()
        .filter(t -> t.getType().equals("INCOME"))
        .mapToDouble(Transaction::getAmount)
        .sum();
}

public double getMonthlyExpenseByCategory(String category, YearMonth month) {
    return transactions.stream()
        .filter(t -> t.getType().equals("EXPENSE"))
        .filter(t -> t.getCategory().equalsIgnoreCase(category))
        .filter(t -> YearMonth.from(t.getDate()).equals(month))
        .mapToDouble(Transaction::getAmount)
        .sum();
}
```

8.3 Budget Manager

Budgets are stored in a HashMap keyed by a composite string category|month. This allows O(1) lookups.

```
private String key(String category, YearMonth month) {
    return category.toLowerCase() + "|" + month;
}

public void setBudget(String category, double limit, YearMonth month) {
    budgets.put(key(category, month), new Budget(category, limit, month));
}
```

8.4 Report Generator

For each budgeted category, the report computes the spent amount, remaining budget, and percentage used. The status and colour are chosen based on those values.

```
if (remaining < 0) {
```

```

        status = RED + "OVER " + formatCurrency(Math.abs(remaining)) + RESET;
        barColor = RED;
    } else if (fraction >= 0.8) {
        status = YELLOW + "WARN" + RESET;
        barColor = YELLOW;
    } else {
        status = GREEN + "OK" + RESET;
        barColor = GREEN;
    }
}

```

8.5 File Storage

Data is saved and loaded as CSV. The `ensureDataDir()` method creates the `data/` and `docs/` folders if they don't exist.

```

public static void save(List<Transaction> transactions) {
    ensureDataDir();
    try (BufferedWriter writer = new BufferedWriter(new FileWriter(TX_FILE))) {
        for (Transaction t : transactions) {
            writer.write(t.toCsv());
            writer.newLine();
        }
    } catch (IOException e) {
        System.out.println("Error saving transactions: " + e.getMessage());
    }
}

```

8.6 Input Validation

Every read operation loops until valid input is provided, so the program cannot crash on malformed input.

```

public static double readDouble(String prompt) {
    while (true) {
        System.out.print(prompt);
        System.out.flush();
        String line = reader.readLine();
    }
}

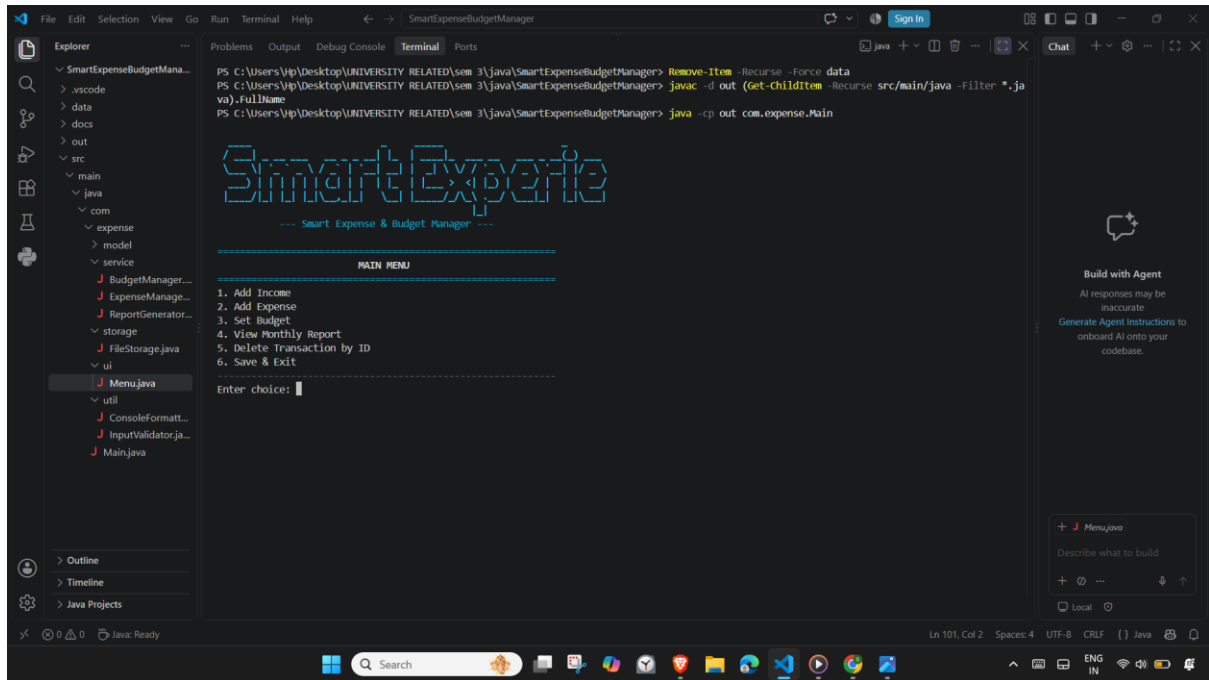
```

```
try {  
    double value = Double.parseDouble(line.trim());  
    if (value < 0) {  
        System.out.println(RED + "Amount cannot be negative." + RESET);  
        continue;  
    }  
    return value;  
} catch (NumberFormatException e) {  
    System.out.println(RED + "Invalid number. Try again." + RESET);  
}  
}  
}
```

9. SCREENSHOTS / RESULTS

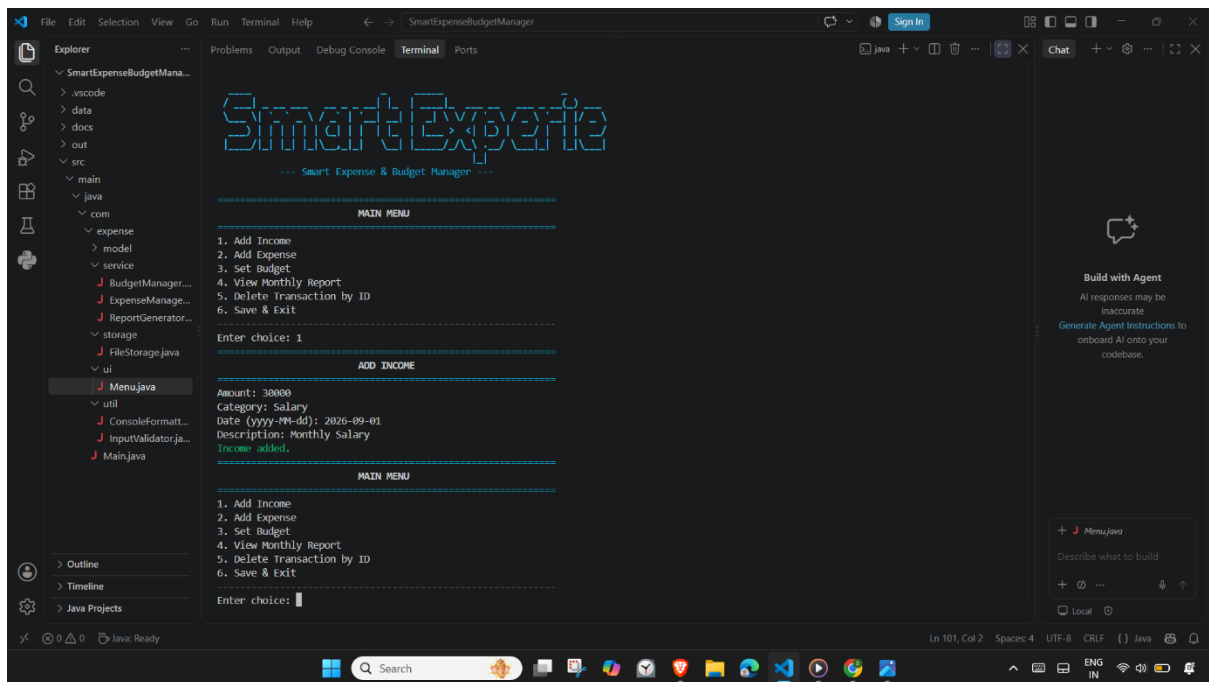
9.1 Main Menu

The application launches with a styled banner and a clear menu.



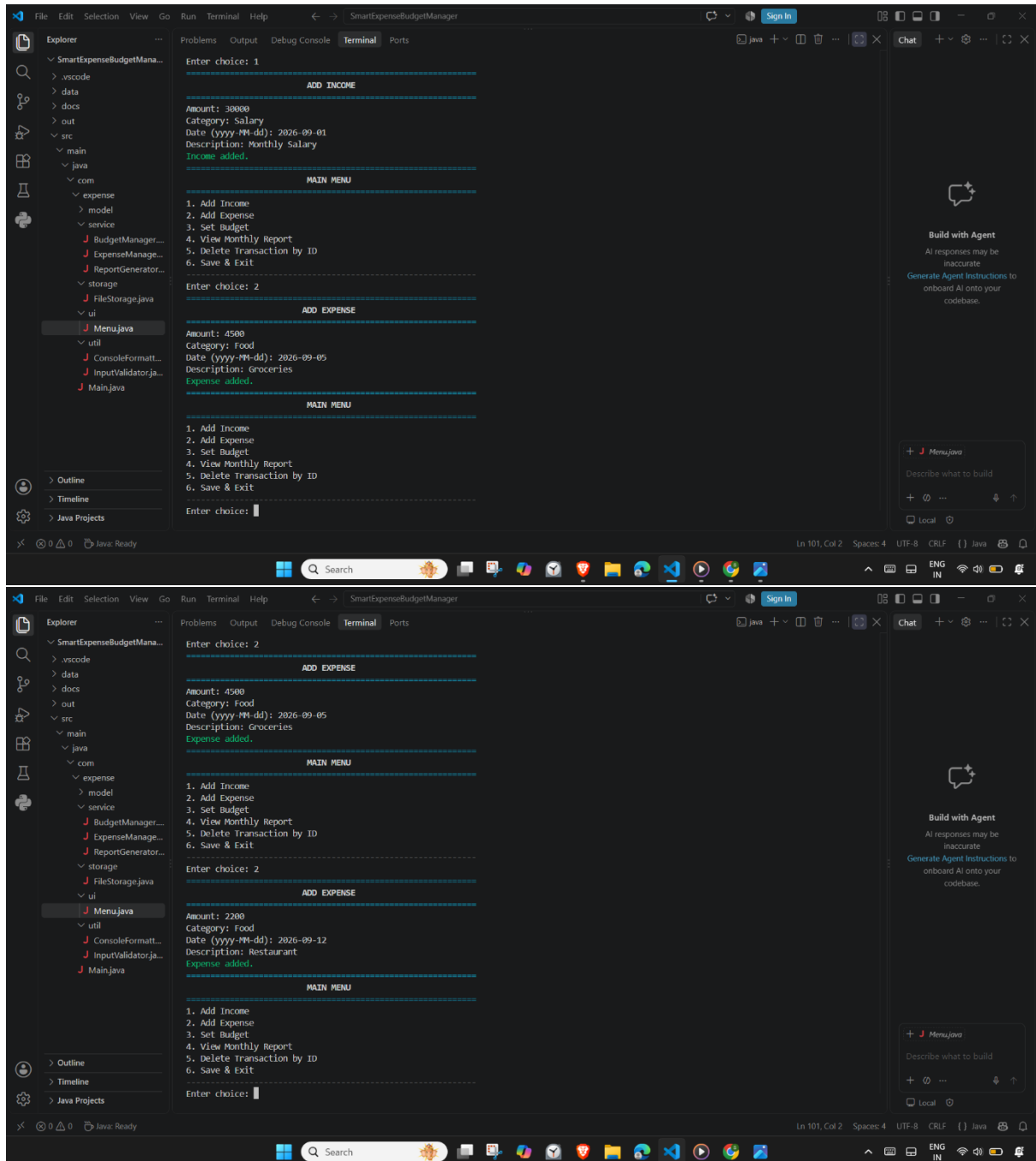
9.2 Adding Income

Income entries are recorded with a category, date, and short description.



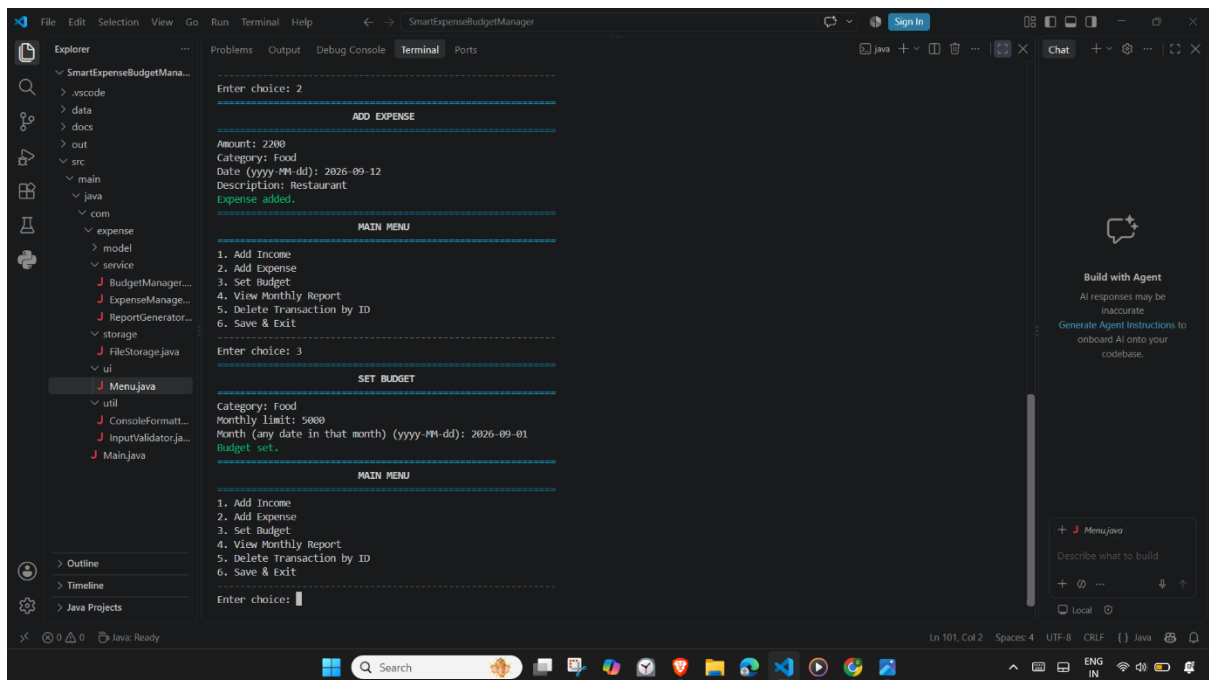
9.3 Adding Expenses

Multiple expenses can be recorded across different categories.



9.4 Setting a Budget

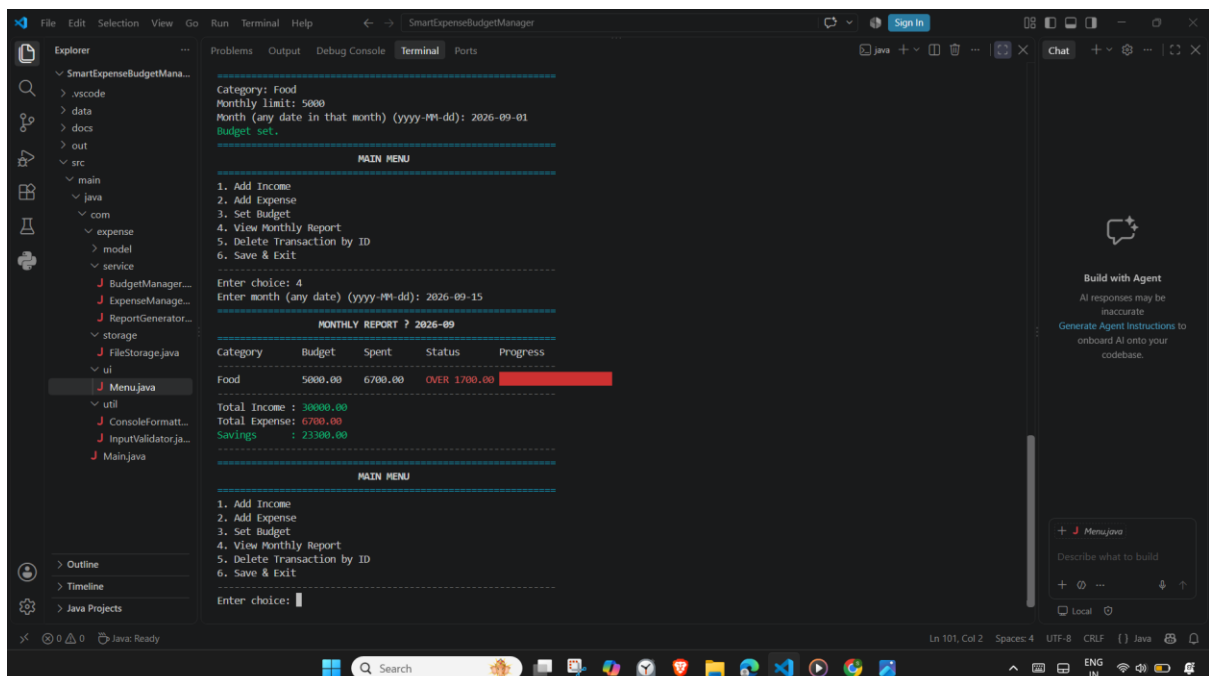
Budgets are set per category per month.



```
Enter choice: 2
=====
ADD EXPENSE
=====
Amount: 2200
Category: Food
Date (yyyy-MM-dd): 2026-09-12
Description: Restaurant
Expense added.
=====
MAIN MENU
=====
1. Add Income
2. Add Expense
3. Set Budget
4. View Monthly Report
5. Delete Transaction by ID
6. Save & Exit
=====
Enter choice: 3
=====
SET BUDGET
=====
Category: Food
Monthly limit: 5000
Month (any date in that month) (yyyy-MM-dd): 2026-09-01
Budget set.
=====
MAIN MENU
=====
1. Add Income
2. Add Expense
3. Set Budget
4. View Monthly Report
5. Delete Transaction by ID
6. Save & Exit
=====
Enter choice: 
```

9.5 Monthly Report

The report shows budget vs. actual spend, with colour-coded status and a progress bar. Categories that exceed their budget are flagged in red.



```
Category: Food
Monthly limit: 5000
Month (any date in that month) (yyyy-MM-dd): 2026-09-01
Budget set.
=====
MAIN MENU
=====
1. Add Income
2. Add Expense
3. Set Budget
4. View Monthly Report
5. Delete Transaction by ID
6. Save & Exit
=====
Enter choice: 4
Enter month (any date) (yyyy-MM-dd): 2026-09-15
=====
MONTHLY REPORT ? 2026-09
=====
Category  Budget  Spent  Status  Progress
=====
Food      5000.00 6700.00 OVER 1700.00
=====
Total Income : 30000.00
Total Expense: 6700.00
Savings      : 23300.00
=====
MAIN MENU
=====
1. Add Income
2. Add Expense
3. Set Budget
4. View Monthly Report
5. Delete Transaction by ID
6. Save & Exit
=====
Enter choice: 
```

9.6 Deleting a Transaction

Transactions can be removed by ID. Attempting to delete a non-existent ID produces a clear error without crashing.

```
File Edit Selection View Go Run Terminal Help SmartExpenseBudgetManager
Explorer
  SmartExpenseBudgetMana...
  .vscode
  > data
  > docs
  > out
  > src
  > main
  > java
  > com
  > expense
  > model
  > service
  > BudgetManager...
  > ExpenseManage...
  > ReportGenerator...
  > storage
  > FileStorage.java
  > ui
  > Menu.java
  > util
  > ConsoleFormat...
  > InputValidator.ja...
  > Main.java
  > Outline
  > Timeline
  > Java Projects
  > Run
  > Debug Console
  > Problems
  > Output
  > Terminal
  > Ports
  > Chat
  > Local

Enter month (any date) (yyyy-MM-dd): 2026-09-15

MONTHLY REPORT ? 2026-09
-----
Category Budget Spent Status Progress
-----
Food 5000.00 6700.00 OVER 1700.00
-----
Total Income : 30000.00
Total Expense: 6700.00
Savings : 23300.00
-----

MAIN MENU
-----
1. Add Income
2. Add Expense
3. Set Budget
4. View Monthly Report
5. Delete Transaction by ID
6. Save & Exit
-----
Enter choice: 5

DELETE TRANSACTION
-----
Enter transaction ID to delete: 4
No transaction found with ID 4.
-----

MAIN MENU
-----
1. Add Income
2. Add Expense
3. Set Budget
4. View Monthly Report
5. Delete Transaction by ID
6. Save & Exit
-----
Enter choice: 
```

9.7 Save and Exit

All data is written to CSV before the program exits.

```
File Edit Selection View Go Run Terminal Help SmartExpenseBudgetManager
Explorer
  SmartExpenseBudgetMana...
  .vscode
  > data
  > docs
  > out
  > src
  > main
  > java
  > com
  > expense
  > model
  > service
  > BudgetManager...
  > ExpenseManage...
  > ReportGenerator...
  > storage
  > FileStorage.java
  > ui
  > Menu.java
  > util
  > ConsoleFormat...
  > InputValidator.ja...
  > Main.java
  > Outline
  > Timeline
  > Java Projects
  > Run
  > Debug Console
  > Problems
  > Output
  > Terminal
  > Ports
  > Chat
  > Local

Enter month (any date) (yyyy-MM-dd): 2026-09-15

MONTHLY REPORT ? 2026-09
-----
Category Budget Spent Status Progress
-----
Food 5000.00 6700.00 OVER 1700.00
-----
Total Income : 30000.00
Total Expense: 6700.00
Savings : 23300.00
-----

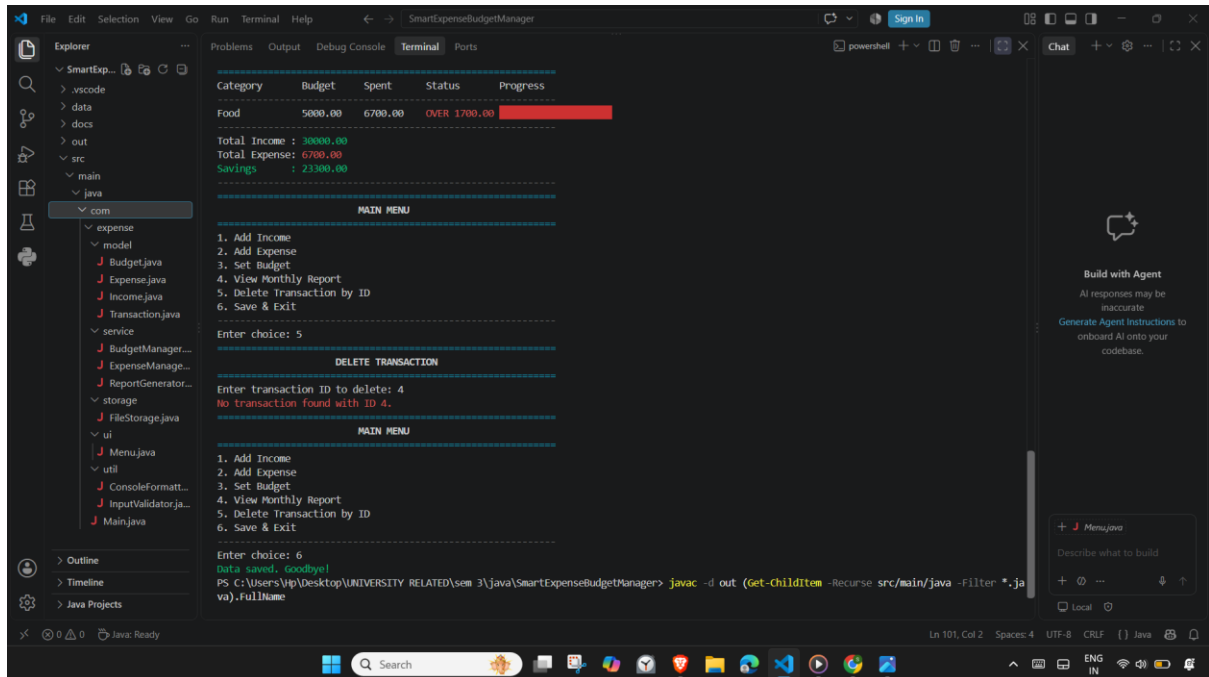
MAIN MENU
-----
1. Add Income
2. Add Expense
3. Set Budget
4. View Monthly Report
5. Delete Transaction by ID
6. Save & Exit
-----
Enter choice: 5

DELETE TRANSACTION
-----
Enter transaction ID to delete: 4
No transaction found with ID 4.
-----

MAIN MENU
-----
1. Add Income
2. Add Expense
3. Set Budget
4. View Monthly Report
5. Delete Transaction by ID
6. Save & Exit
-----
Enter choice: 6
Data saved. Goodbye!
PS C:\Users\vip\Desktop\UNIVERSITY-RELATED\sem 3\java\SmartExpenseBudgetManager>
```

9.8 Compilation

The project compiles cleanly from a single javac command with no errors.



```
PS C:\Users\vip\Desktop\UNIVERSITY RELATED\sem 3\java\SmartExpenseBudgetManager> javac -d out (get-ChildItem -Recurse src/main/java -filter *.java).FullName

Category    Budget    Spent    Status    Progress
-----
Food        5000.00   6700.00   OVER 1700.00   ██████████

Total Income : 30000.00
Total Expense: 6700.00
Savings      : 23300.00

=====
MAIN MENU
=====
1. Add Income
2. Add Expense
3. Set Budget
4. View Monthly Report
5. Delete Transaction by ID
6. Save & Exit

Enter choice: 5

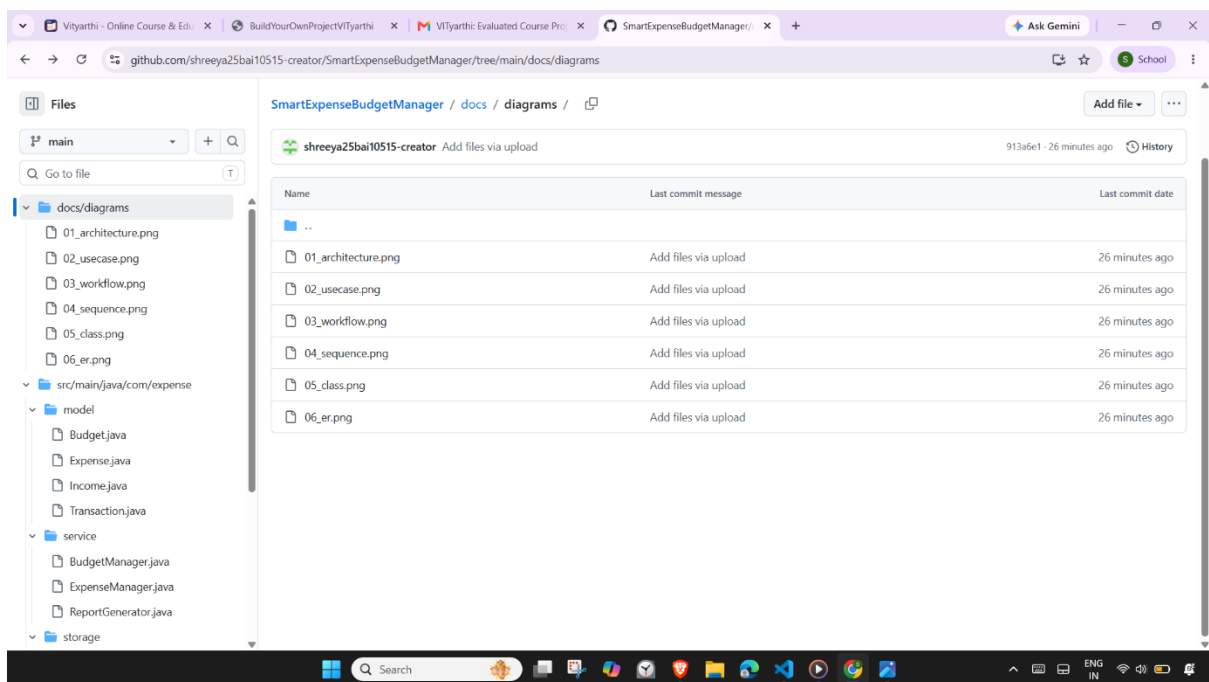
=====
DELETE TRANSACTION
=====
Enter transaction ID to delete: 4
No transaction found with ID 4.

=====
MAIN MENU
=====
1. Add Income
2. Add Expense
3. Set Budget
4. View Monthly Report
5. Delete Transaction by ID
6. Save & Exit

Enter choice: 6
Data saved. Goodbye!
```

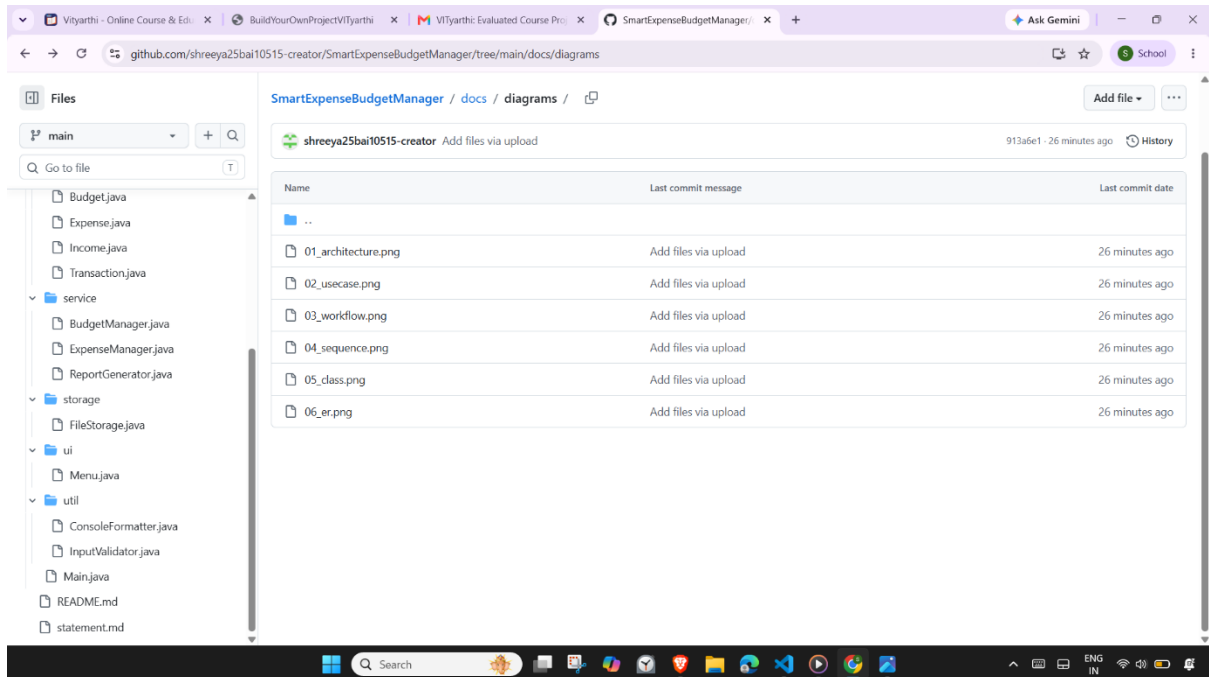
9.9 Project Structure

The project follows a clean layered package structure.



9.10 GitHub Repository

The complete source code, README, project statement, and design diagrams are hosted on a public GitHub repository.



10. TESTING APPROACH

The application was tested manually across all menu paths. Because the project does not use a formal test framework, testing focused on validation testing — deliberately feeding invalid input to each prompt to confirm the program handles it gracefully.

Test cases executed:

Test 1

Input: Amount 30000, Salary, 2026-09-01

Expected Behaviour: Entry added, appears in report

Result: Pass

Test 2

Input: Amount 4500, Food, 2026-09-05

Expected Behaviour: Entry added, appears in report

Result: Pass

Test 3

Input: Food, 5000, 2026-09-01

Expected Behaviour: Budget saved, appears in report

Result: Pass

Test 4

Input: Expenses exceeding 5000 in Food

Expected Behaviour: Status shows OVER in red

Result: Pass

Test 5

Input: Spend at 80%+ of budget

Expected Behaviour: Status shows WARN in yellow

Result: Pass

Test 6

Input: ID 2

Expected Behaviour: Row removed

Result: Pass

Test 7

Input: ID 999

Expected Behaviour: Error message, no crash

Result: Pass

Test 8

Input: "abc"

Expected Behaviour: Re-prompt, no crash

Result: Pass

Test 9

Input: "-500"

Expected Behaviour: Rejected, re-prompt

Result: Pass

Test 10

Input: "15/09/2026"

Expected Behaviour: Rejected, re-prompt

Result: Pass

Test 11

Input: Blank line

Expected Behaviour: Rejected, re-prompt

Result: Pass

Test 12

Input: "99"

Expected Behaviour: "Invalid choice" message

Result: Pass

Test 13

Input: Save, close, reopen

Expected Behaviour: Data still present

Result: Pass

Test 14

Input: Delete data folder, run app

Expected Behaviour: Folder recreated

Result: Pass

All test cases passed. No crashes were observed during testing.

11. CHALLENGES FACED

Terminal input handling on Windows

The original implementation used `Scanner.nextInt()` and `nextLine()` in sequence, which caused a well-known bug where the newline character was consumed unexpectedly. This was resolved by switching to `BufferedReader.readLine()` and parsing each line explicitly. This made input handling deterministic across Windows PowerShell, CMD, and Unix shells.

Persisting budgets alongside transactions

The initial design only saved transactions. Budgets were lost on every exit. The fix was to add a second CSV file (`budgets.csv`) and a pair of symmetric methods `saveBudgets()` / `loadBudgets()` in the `FileStorage` class.

ANSI colour support in different terminals

Some terminals do not interpret ANSI escape codes correctly, showing raw codes instead of colours. To keep the output readable either way, the design ensures that even when colours do not render, the text is still well-aligned and clear.

Progress bar rendering

Rendering a clean bar chart in a terminal required careful use of Unicode block characters (■ and ▒) plus integer rounding of the fraction. Edge cases like a zero budget or over 100% usage were handled explicitly to avoid visual glitches.

Accidental auto-hints in VS Code

While writing code, VS Code's inline parameter hints were occasionally mistaken for actual syntax, causing compilation errors. Disabling inline hints resolved this during development.

12. LEARNINGS & KEY TAKEAWAYS

Layered architecture pays off

Once the project was organised into packages with single responsibilities, adding new features and fixing bugs became dramatically easier. Small changes stayed small.

Input validation is non-negotiable

More time was spent fixing crashes caused by malformed input than by any other category of bug. Centralising all input handling in one utility class eliminated the problem.

Streams make aggregate code readable

Computing totals, filtering by category, and summing by month is much cleaner with streams than with manual loops. The intent of the code is now visible at a glance.

Persistence is a design decision, not an afterthought

Adding CSV persistence midway through development required changes across several classes. A better plan would have been to include it in the architecture from the start.

Small details matter for usability

Colour-coded output and progress bars added only a few dozen lines of code, but they significantly changed how pleasant the application is to use.

README-first development

Writing the README early forced clarity about what the application actually does and how it should be run. This in turn surfaced gaps in the design.

13. FUTURE ENHANCEMENTS

If development continued, the following features would be natural additions:

1. Export reports to PDF or text file — useful for long-term record keeping.
2. Recurring transactions — automatically add rent or salary each month.
3. Month-over-month comparison — show trends in spending.
4. Multi-user support — separate data files per user.
5. JUnit test suite — formalise the manual testing already performed.
6. Configuration file — let the user predefine categories instead of typing them freely.
7. Search and filter — find transactions by date range or category.
8. Encryption — protect the CSV files with a password.
9. Import from bank statements — parse a CSV export from a bank.
10. Graphical interface — using JavaFX, for users who prefer a visual UI.

12. REFERENCES

1. Oracle. Java SE 21 Documentation.
<https://docs.oracle.com/en/java/javase/21/>
2. Oracle. Java Stream API.
<https://docs.oracle.com/javase/8/docs/api/java/util/stream/package-summary.html>
3. Oracle. Java Date and Time API (java.time).
<https://docs.oracle.com/javase/tutorial/datetime/>
4. Baeldung. Reading and Writing Files in Java.
<https://www.baeldung.com/java-write-to-file>
5. GitHub Docs. About READMEs.
<https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-readmes>
6. Mermaid. Class Diagram Syntax.
<https://mermaid.js.org/syntax/classDiagram.html>